



AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

Faculty of Engineering

Lab Report

Experiment # 05

Experiment Title: Familiarization of assembly language program and Interrupts in a microcontroller.

Date of Perform:	06 Nov 2025	Date of Submission:	15 Nov 2025
Course Title:	MICROPROCESSOR AND EMBEDDED SYSTEMS LAB		
Course Code:	EEE 4103	Section: [O]	
Semester:	Fall 2025-26	Degree Program:	BSc in CSE
Course Teacher:	Dr. Md. Rukonuzzaman		

Declaration and Statement of Authorship:

1. I/we hold a copy of this Assignment/Case Study, which can be produced if the original is lost/damaged.
2. This Assignment/Case Study is my/our original work and no part of it has been copied from any other student's work or from any other source except where due acknowledgment is made.
3. No part of this Assignment/Case Study has been written for me/us by any other person except where such collaboration has been authorized by the concerned teacher and is acknowledged in the assignment.
4. I/we have not previously submitted or currently submitting this work for any other course/unit.
5. This work may be reproduced, communicated, compared, and archived to detect plagiarism.
6. I/we permit a copy of my/our marked work to be retained by the Faculty Member for review by any internal/external examiners.
7. I/we understand that Plagiarism is the presentation of the work, idea, or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offense that may lead to expulsion from the University. Plagiarized material can be drawn from, and presented in, written, graphic, and visual forms, including electronic data, and oral presentations. Plagiarism occurs when the origin of the source is not appropriately cited.
8. I/we also understand that enabling plagiarism is the act of assisting or allowing another person to plagiarize or copy my/our work.

* Student(s) must complete all details except the faculty use part.

** Please submit all assignments to your course teacher or the office of the concerned teacher.

Group # 06

Sl No	Name	ID	PROGRAM	SIGNATURE
1	Mohammad Ansar Uddin	22-47975-2	BSc in CSE	
2	Chinmoy Guha	22-48056-2	BSc in CSE	
3	Nafiur Rahman Siam	23-50834-1	BSc in CSE	
4	Suvra Chakraborty	22-48067-2	BSc in CSE	
5	Arfanul Hoque Nehal	23-54270-3	BSc in CSE	
6	Mashkur Ibtesham Wazed	23-54266-3	BSc in CSE	

Faculty use only

FACULTY COMMENTS	Marks Obtained	
	Total Marks	

Table of Contents

Experiment Title	3
Objectives	3
Equipment List	3
Circuit Diagram	4
Code/Program	5
Hardware Output Results	6
Experimental Output Results	7
Simulation Output Results	8
Answers to the Questions in the Lab Manual	9
Discussion	12
References	13

Experiment Title: Familiarization of assembly language program and Interrupts in a microcontroller.

Objectives:

The objectives of this experiment are to

1. Study the assembly language program of an Arduino.
2. Write assembly language programming code for an Arduino.
3. Build a circuit to turn on and off an LED on an Arduino Microcontroller Board connected to an I/O port of the microcontroller.
4. Study of the external interrupts of an Arduino using its digital I/O port.
5. Build a circuit to turn on and off an LED on an Arduino Microcontroller Board connected to an I/O port of the microcontroller due to the external interrupt.

Theory and Methodology:

In this experiment, the theoretical foundation and the practical methodology are tightly coupled around two core ideas: assembly-level control of the ATmega328P microcontroller on the Arduino Uno and the use of hardware interrupts for responsive I/O. At the theoretical level, the work begins with understanding that Arduino sketches can be extended with low-level AVR assembly, written in paired files (.ino and .S) that share a common name and reside in the same directory. The ATmega328P provides 32 general-purpose registers and several 8-bit I/O ports (A, B, C and D), each governed by data direction registers that define whether individual bits function as inputs or outputs. Configuring these registers allows precise control over pins, such as setting PB5 (digital pin 13) as an output to drive an LED. Complementing this is the theory of interrupts: asynchronous events generated by internal peripherals (e.g., timers) or external signals (e.g., switches) that temporarily suspend the main program flow. When an interrupt occurs, the processor completes the current instruction, jumps to a specific interrupt vector, executes an Interrupt Service Routine (ISR) without arguments, and then returns to the main program, with registers and status flags preserved so that normal execution can resume without corruption. On the Arduino platform this includes external interrupts on digital pins, timer compare-match interrupts, and pin-change interrupts, each mapped to specific vectors and configured via the relevant control and mask registers.

Methodologically, the experiment implements this theory through a sequence of hardware–software tasks using the Arduino Uno, LEDs, a current-limiting resistor, push switches, and jumper wires. Students first write and debug C and assembly code in the Arduino IDE, creating pairs of files such as led.ino/led.S and btnLED.ino/btnLED.S, then compile and upload the resulting machine code via USB to the microcontroller. The LED blink in Part 1 is achieved by configuring the appropriate data direction register for PB5 and using an assembly delay routine to establish a visible on–off period. Subsequent parts extend the methodology by wiring push buttons to selected digital pins and enabling different interrupt sources: external interrupts to toggle the LED state when a button input changes, a Timer1 compare-match interrupt configured with a prescaler and output compare value to generate periodic LED blinking, and a pin-change interrupt on specific port bits to detect transitions on multiple inputs. Each configuration is validated both on real hardware and, where applicable, through Proteus simulations that load the compiled HEX files, allowing comparison between simulated and physical behaviour. Overall, the combined theory–method approach demonstrates how low-level register programming and interrupt mechanisms can be systematically applied to design a simple but robust LED control system using a microcontroller.

Equipment List:

- 1) Arduino IDE (2.0.1 or any recent version)
- 2) Arduino Microcontroller board
- 3) PC having an Intel processor
- 4) LED lights (Red, Green, Yellow, 1 pc each)
- 5) One 100 Ω resistor
- 6) Three push switches
- 7) Jumper wires

Circuit Diagram:

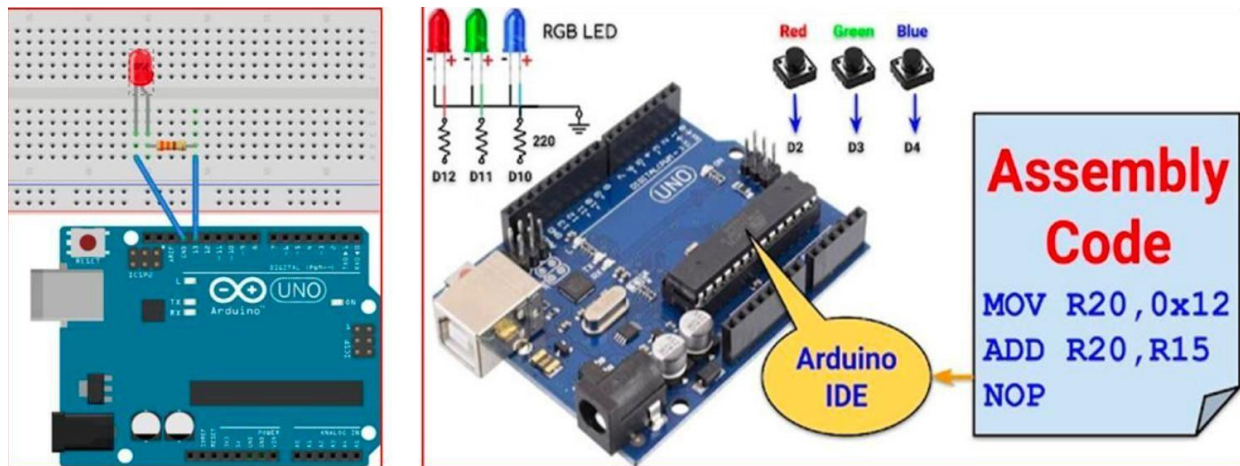


Figure 1: LED control system using an Arduino Microcontroller Board.

Experimental Output Results:

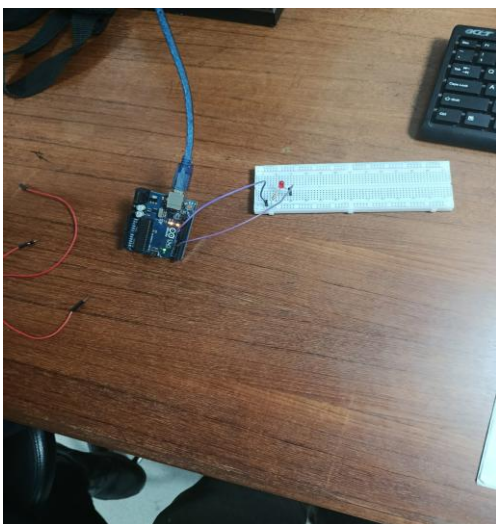


Figure 2: LED is OFF

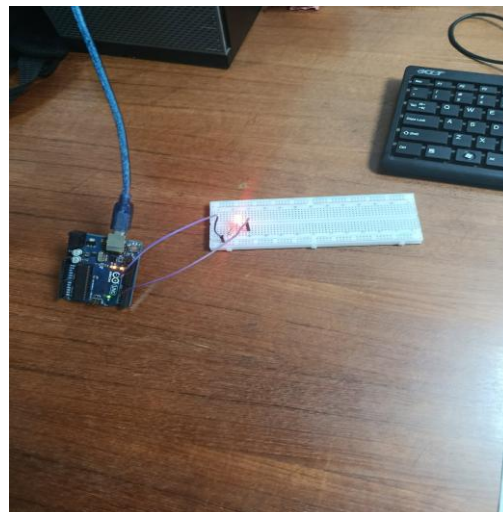


Figure 3: LED is ON

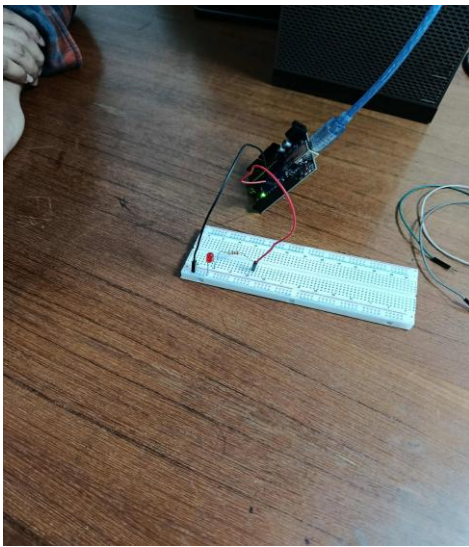


Figure 4: LED is OFF

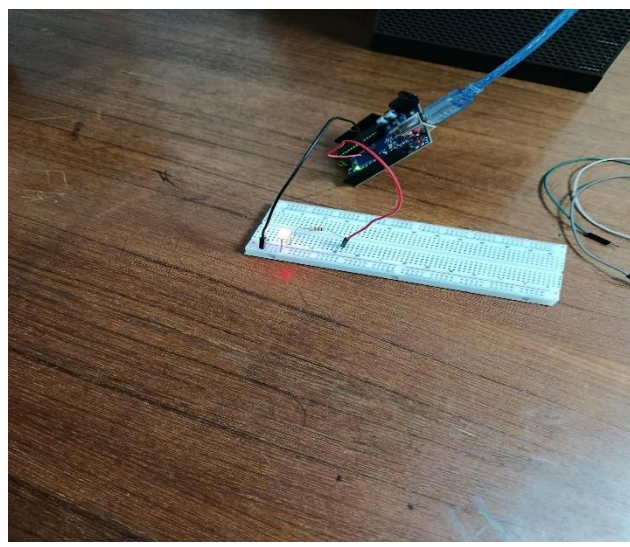


Figure 5: LED is ON, and the push button does not press.

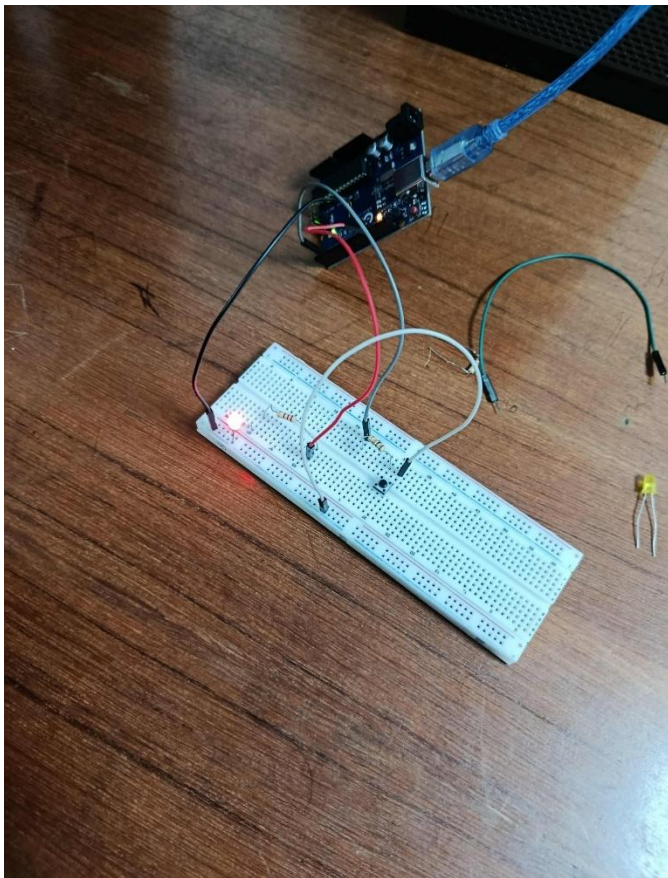


Figure 6: LED changes state when push button is pressed

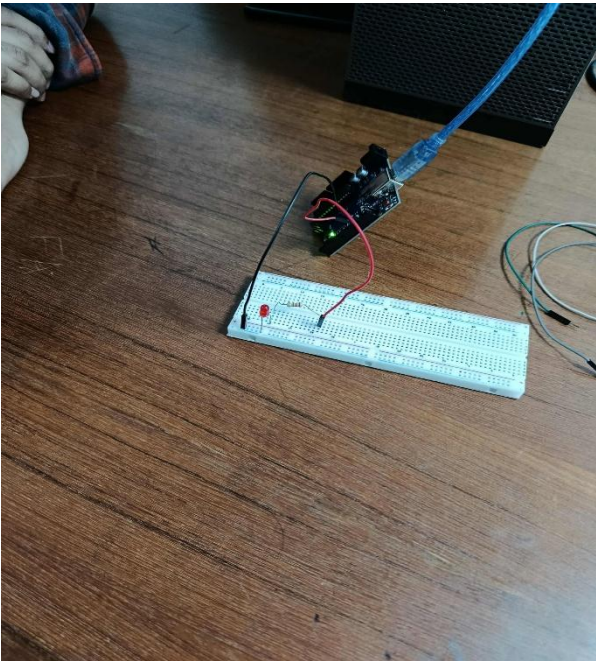


Figure 7: LED is OFF, push button not pressed

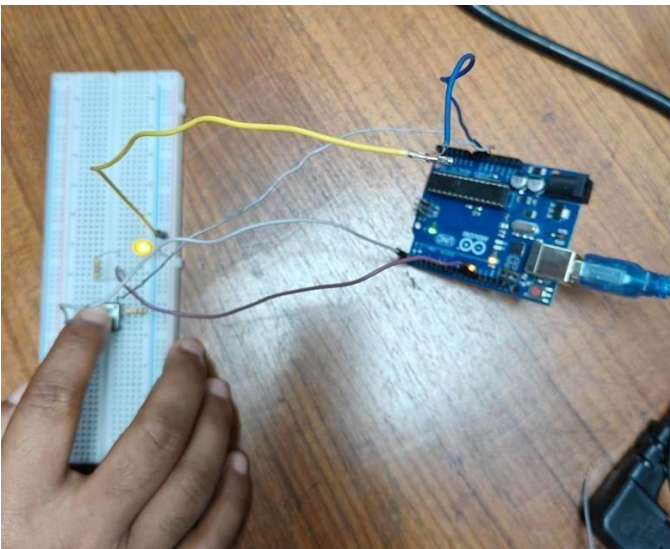


Figure 8: The push button is pressed, and the LED is on.

Simulation Output Results:

Part 1:

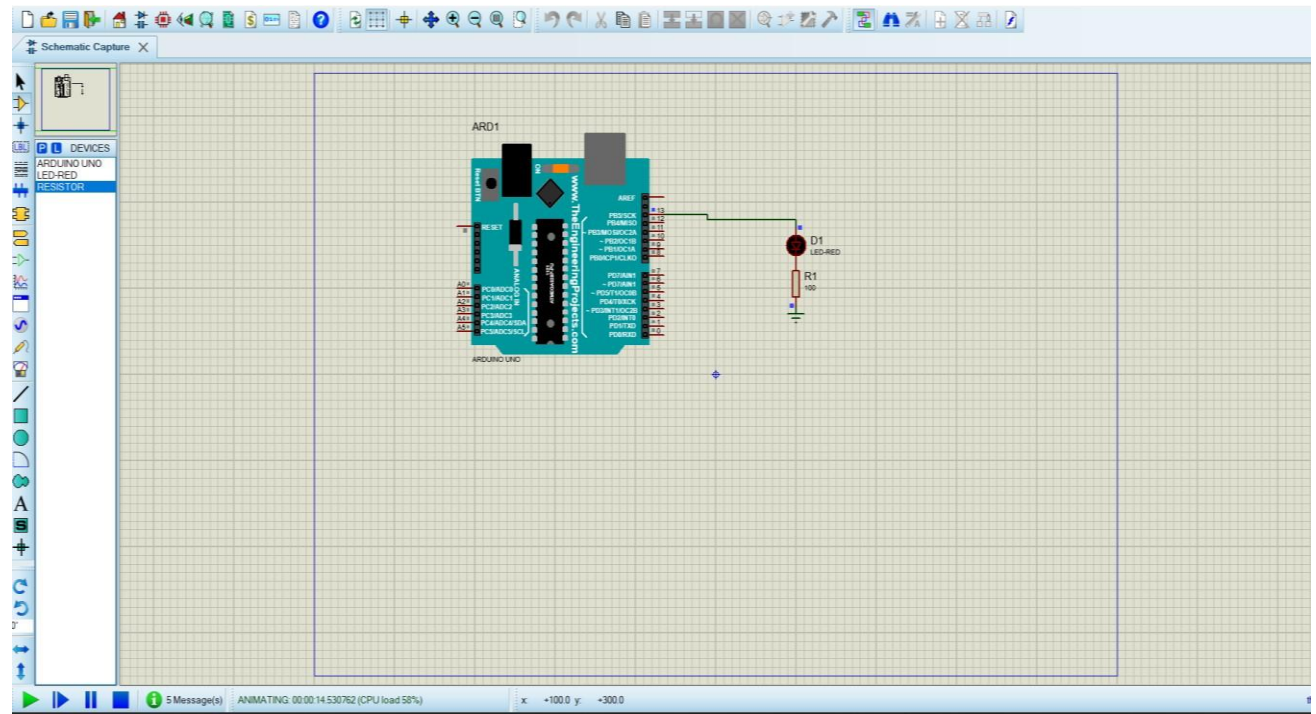


Figure 9: LED OFF

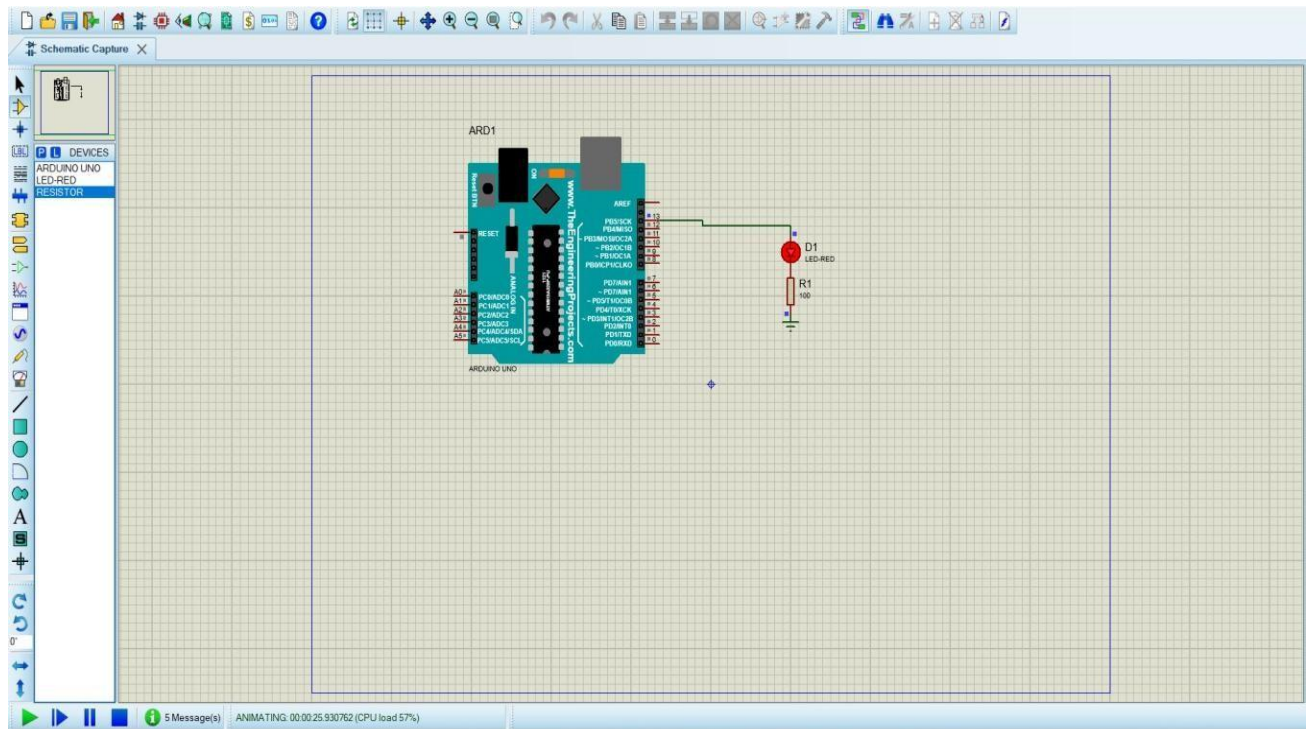


Figure 10: LED ON

Part 2:

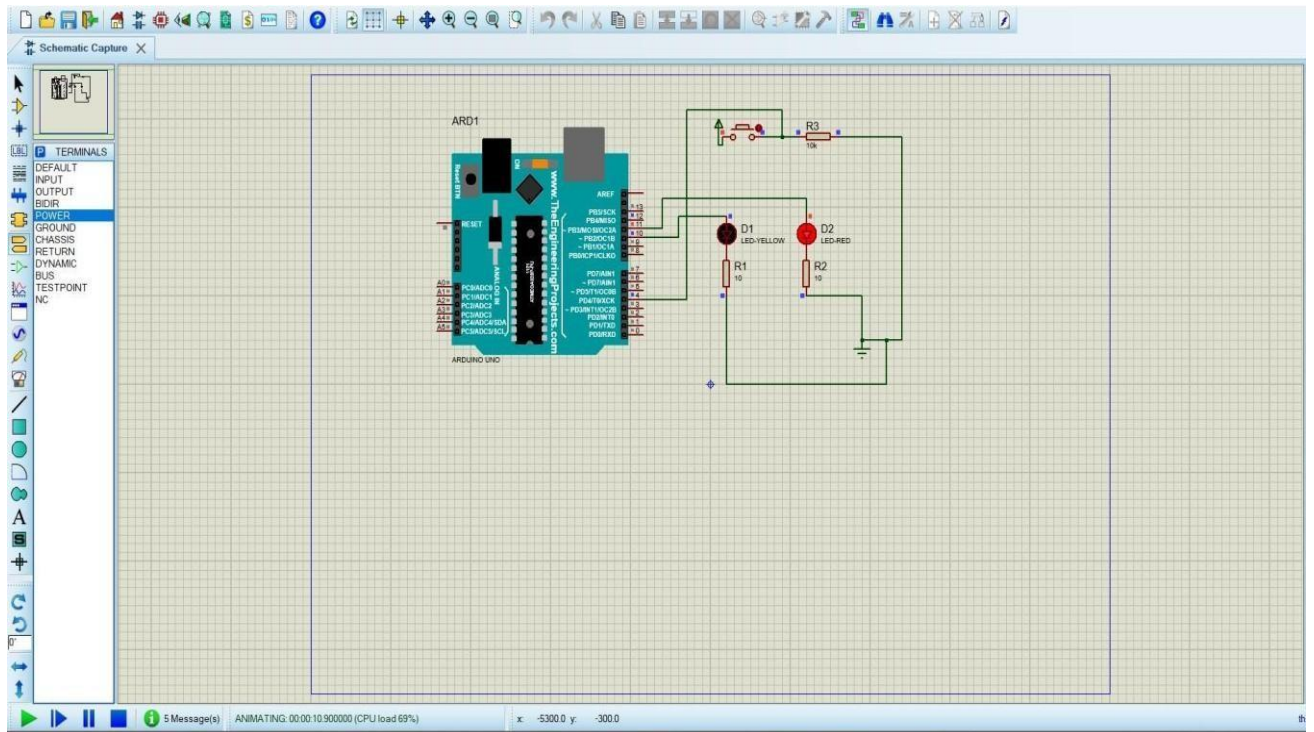


Figure 11: Red LED is on and the yellow LED is OFF when the switch is not pressed

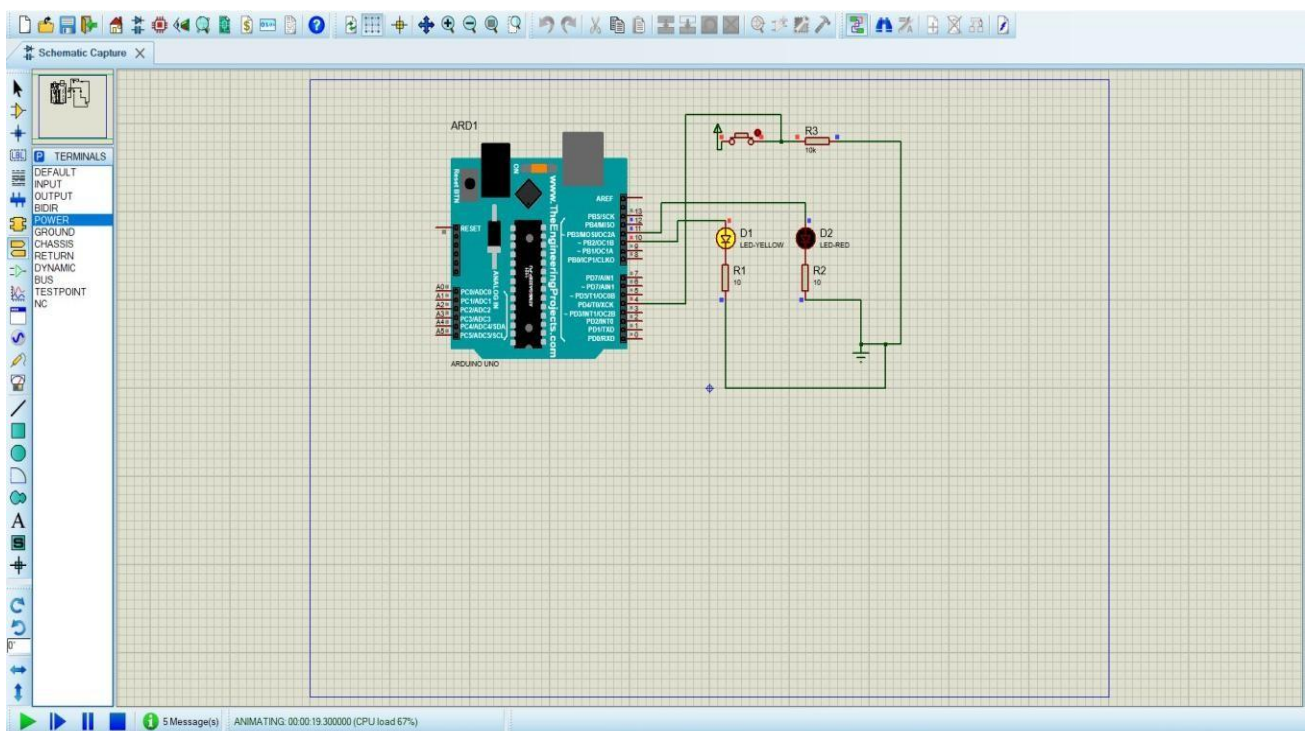


Figure 11: Red LED is off and the yellow LED is ON when the switch is pressed

Part 3 :

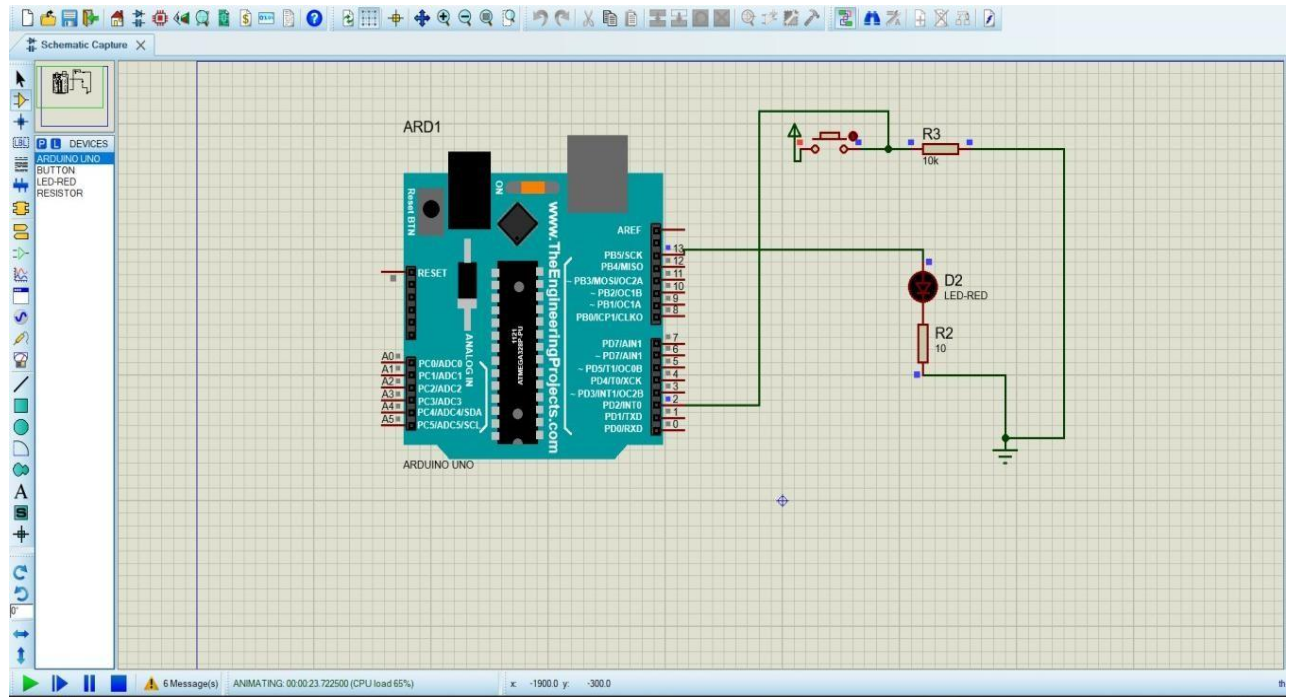


Figure 12: LED is OFF when the switch is not pressed

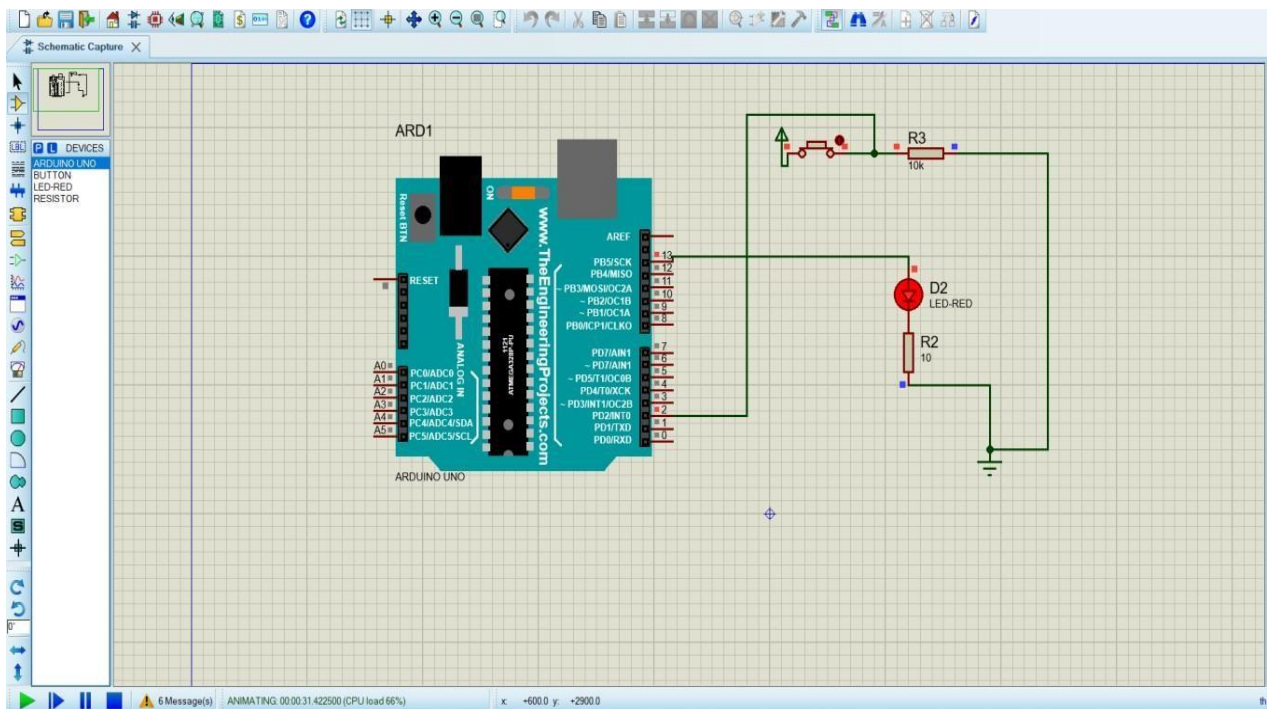


Figure 13: LED is OFF when the switch is pressed.

Part 4:

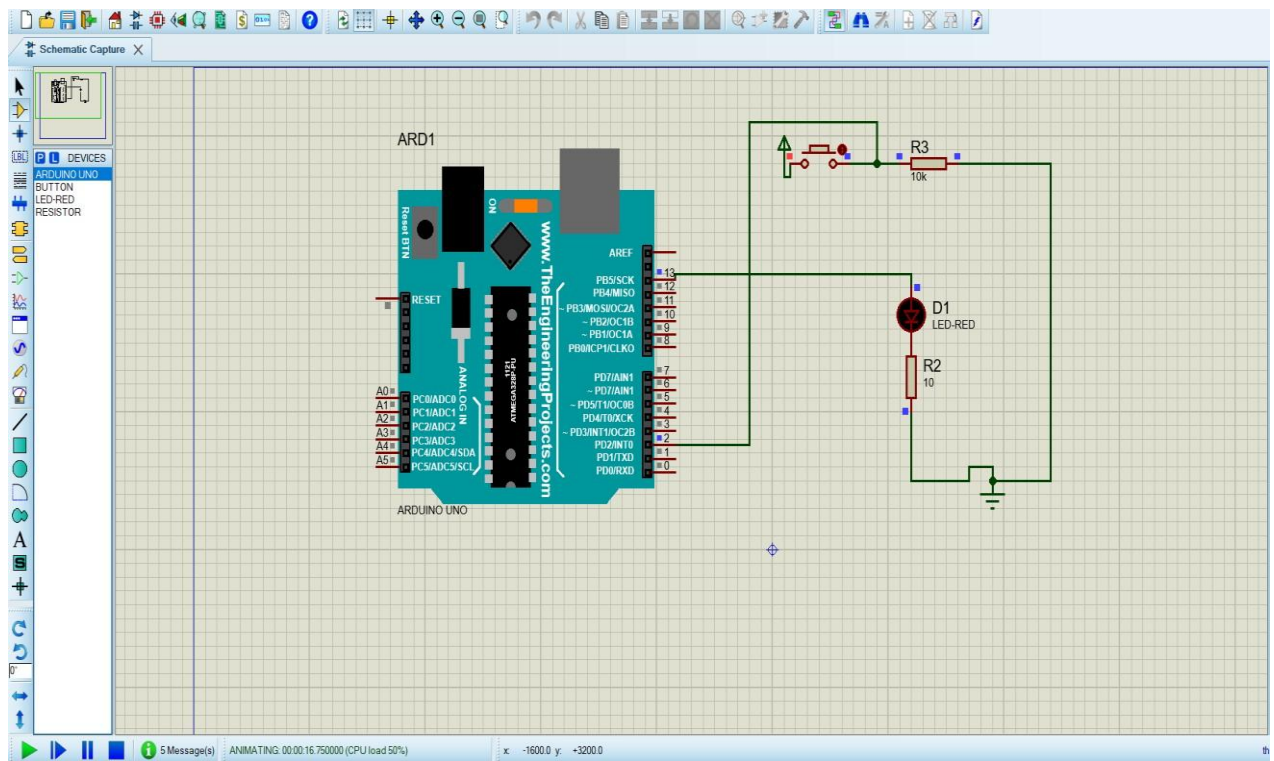


Figure 14: LED is OFF in the blinking LED test

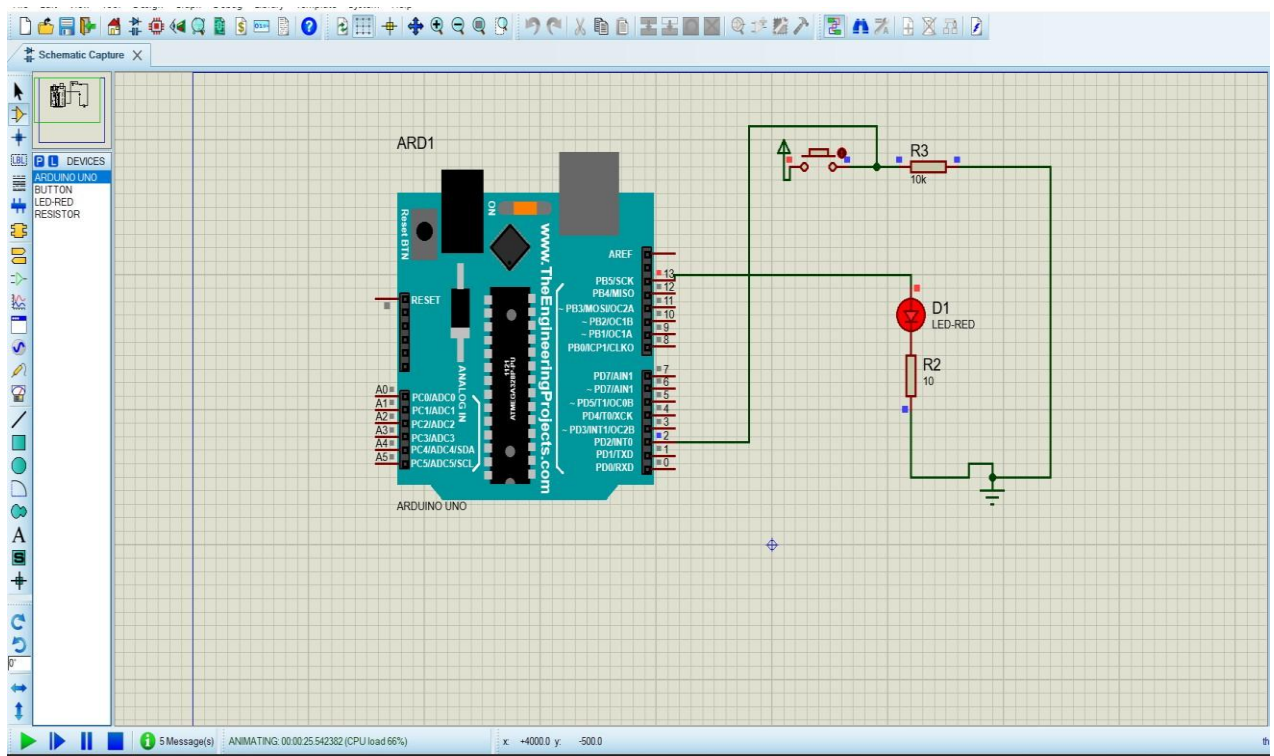


Figure 15: LED is Off in the blinking LED test

Part 5:

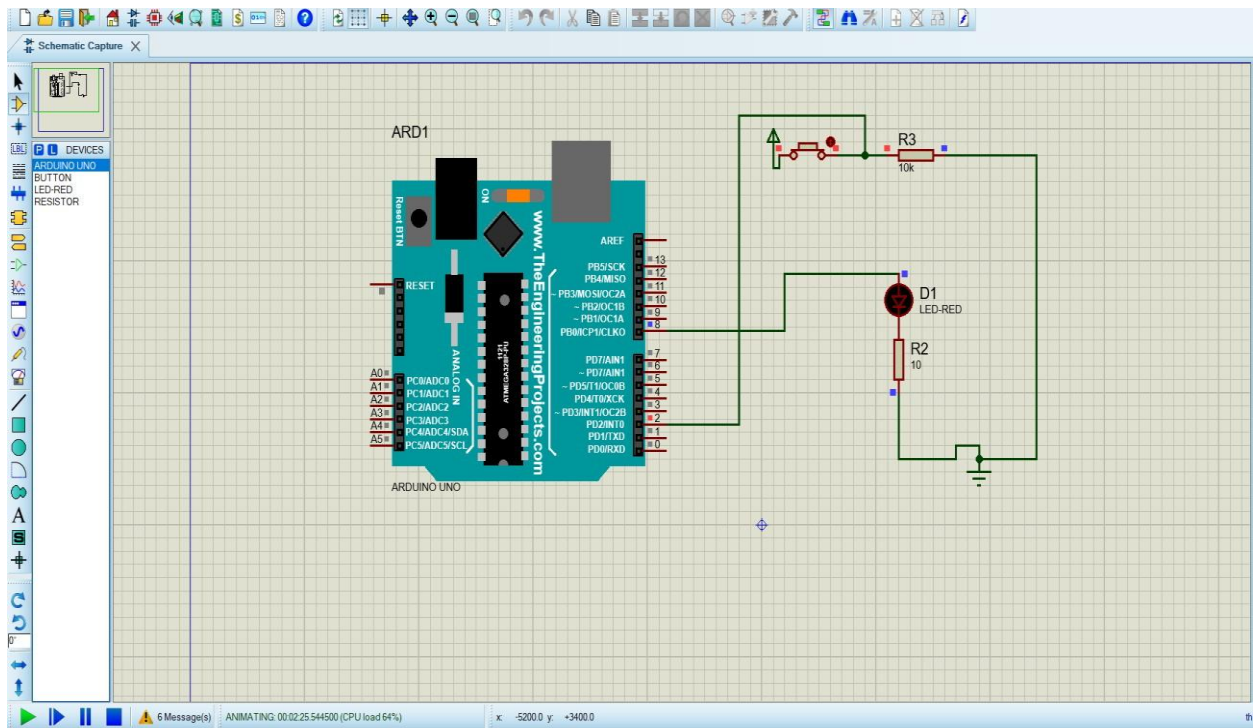


Figure 16: LED is OFF when the switch is not pressed or interrupted.

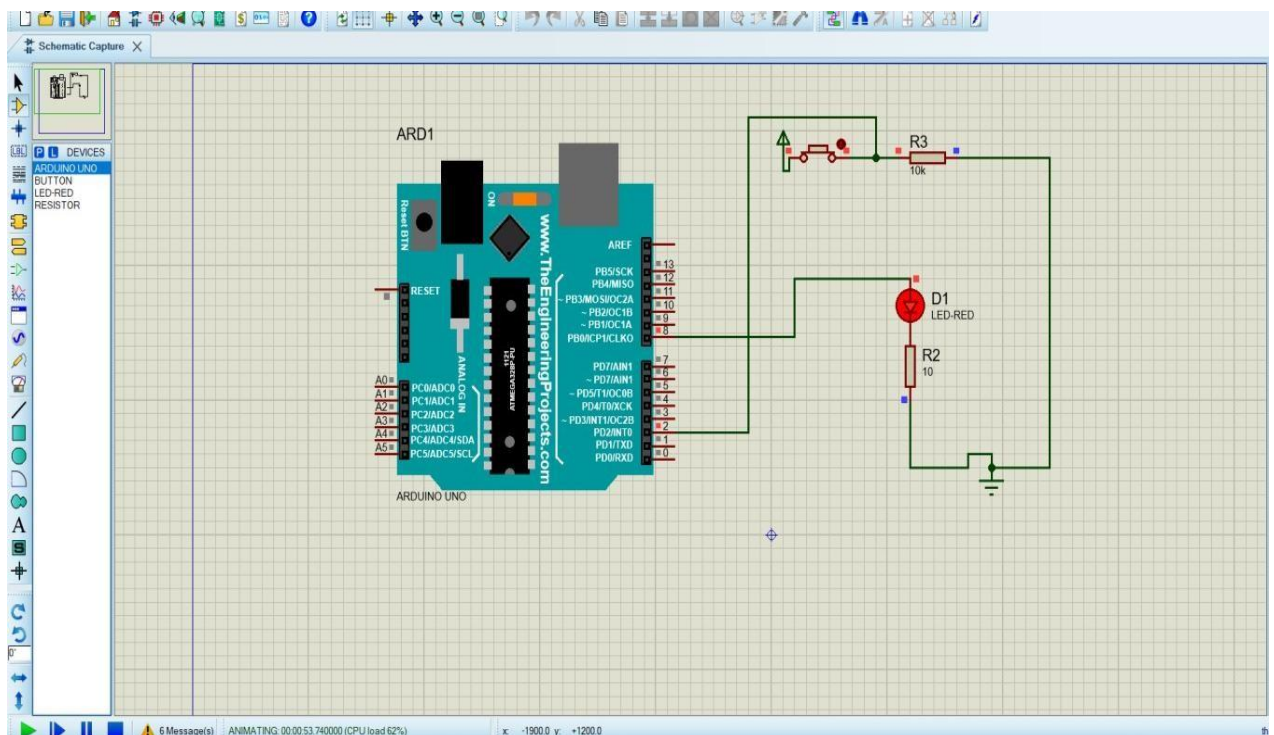


Figure 17: LED is ON when the switch is pressed or interrupted.

Answers to the Questions in the Lab Manual:

Part 1 (Arduino code)

```
//
// C Code for Blinking LED
//
extern "C"
{ void start(); void led(byte);
}
//
void setup()
{ start();
}
//
void loop()
{ led(1); led(0); }
Assembly code
;
; Assembly Code
;
#define ____ SFR_OFFSET 0x00
#include "avr/io.h"
;
.global start
.global led
;
start:
SBI DDRB, 5; set PB5 (D13) as o/p RET;
return to setup() function
;
led:
CPI R24, 0x00; value in R24 passed by caller compared with 0 BREQ ledOFF; jump (branch) if
equal to subroutine ledOFF SBI PORTB, 5;
set D13 to HIGH, i.e., the LED will turn ON
RCALL myDelay; Calling a delay function to determine the ON duration of LED RET; return to
loop() function
;
ledOFF:
CBI PORTB, 5; set D13 to LOW, i.e., the LED will turn OFF
RCALL myDelay; Calling a delay function to determine the OFF duration of LED RET; return to
loop() function
;
.equ delayVal, 10000; initial count value for the inner loop (0010011100010000)
;
myDelay:
LDI R20, 100; initial count value for the outer loop outerLoop:
LDI R30, lo8(delayVal); LDI R31, hi8(delayVal); low byte of delayVal in R30 (00010000) high
byte of delayVal in R31 (00100111) innerLoop:
SBIW R30, 1; BRNE innerLoop; subtract 1 from 16-bit value in R31, R30 jump if countVal not
equal to 0
;
SUBI R20, 1; BRNE outerLoop; subtract 1 from R20 jump if
R20 is not equal to 0
RET
;
```

Part 2 (Arduino code)

```
//  
// C Code: RGB LED ON/OFF via Buttons  
//  
extern "C"  
{ void start(); void  
btnLED();  
}  
//  
void setup()  
{ start();  
}  
//  
void loop()  
{ btnLED();  
}
```

Assembly code:

```
//Assembly Code: RGB LED ON/OFF via Buttons  
//  
#define __ SFR_OFFSET 0x00  
#include "avr/io.h"  
;  
.global start  
.global btnLED start:  
;===== SBI SBI SBI CBI CBI  
CBI RET  
DDRB, 4; DDRB, 3; DDRB, 2; DDRD, 2; DDRD, 3; DDRD, 4; set PB4 (pin D12 as o/p - red  
LED) set PB3 (pin D11 as o/p - green LED) set PB2 (pin D10 as o/p - blue LED) clear  
PD2  
(pin D02 as i/p - red button) clear PD3 (pin D03 as i/p - green button) clear PD4  
(pin D04 as i/p - blue button) btnLED:  
L2: SBIS PIND, 4 RJMP L1  
SBI PORTB, 2 CBI PORTB, 3 SBIC PIND, 4 RJMP L2  
; Skips below statement if the push button of D04 is not pressed  
; Turn ON LED, PB2(D10), if SW of D04 is not pressed  
; Turn OFF LED, PB3(D11), if SW of D04 is not pressed  
; Skips below statement if the push button of D04 is pressed L1: CBI  
SBI RET  
PORTB, 2 PORTB, 3 ; Turn OFF LED, PB2(D10), if SW of D04 is pressed ; Turn OFF LED,  
PB3(D11), if SW of D04 is pressed
```

Part 3

```
const byte ledPin = 13; const byte interruptPin = 2; volatile byte state = LOW;
void setup() { pinMode(ledPin, OUTPUT);
pinMode(interruptPin, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(interruptPin), blink,
CHANGE);
} void loop() { digitalWrite(ledPin, state);
} void blink() { state = !state;
}
```

Part 4

```
bool LED_State = 'True';
void setup() { pinMode(13, OUTPUT);
cli(); // stop interrupts till we make the settings
TCCR1A = 0; // Reset the entire A and B registers of Timer1 to make sure that TCCR1B
= 0; // we start with everything disabled.
TCCR1B = 0b00000100; // Set CS12 bit of TCCR1B to 1 to get a prescalar value of 256.
TIMSK1 = 0b00000010; // Set OCIE1A bit to 1 to enable compare match mode of A reg.
OCR1A = 31250; // We set the required timer count value in the compare register, A
sei(); // Enable back the interrupts void loop() {
}
}
// put your main code here, to run repeatedly.
// With the settings above, this ISR will trigger each 500 ms.
ISR(TIMER1_COMPA_vect) {
TCNT1 = 0; // First, set the timer back to 0 so that it resets for the next interrupt
LED_State =
!LED_State; // Invert the LED State
digitalWrite(13, LED_State); // Write this new state to the LED connected to pin D5
}
```

Part 5

```
#include "PinChangeInterrupt.h"
#define BTN
#define LED

PIN 8
PIN 7

void BTN_ISR(void)
{
digitalWrite(LED_PIN, !digitalRead(LED_PIN));
}
void setup() { pinMode(LED_PIN, OUTPUT);
pinMode(BTN_PIN, INPUT); attachPCINT(digitalPinToPCINT(BTN_PIN), BTN_ISR, RISING);
} void loop() { // Do Nothing
}
```


Discussion:

This experiment explored the use of an Arduino Uno Microcontroller board to control an LED. The setup replicated the hardware configuration previously implemented, which included connecting the anode of the LED to PIN 13 of the microcontroller and introducing a 100-ohm resistor in series with the LED's cathode. This resistor was then linked to the microcontroller's GND (Ground) pin. Creating and verifying the code in the Arduino IDE was executed successfully, leading to generating a HEX file. This file contained the instructions necessary for the simulation. Subsequently, the Proteus simulation environment was employed to execute the instructions encapsulated in the HEX file, mimicking the operation of the Arduino Uno and the LED. The simulation was completed, and the results obtained were methodically recorded and compared with the outcomes of the hardware implementation.

The experiment highlighted the practicality and utility of simulations in testing and verifying hardware-related functionalities. It provided valuable insights into the consistency of results between real-world hardware and virtual simulations, enhancing our understanding of microcontroller-based systems.

References:

- [1] <https://www.arduino.cc/>
- [2] ATmega328 manual
- [3] <https://www.avrfreaks.net/forum/tut-c-newbies-guide-avr-0mers>
- [4] <http://maxembedded.com/2011/06/avr-0mers-0mer0/>